Auditable Compositional Question Answering over UK Public Procurement

Anonymous *anonymous@example.com*
University College London

Abstract

UK public procurement records are public and partly structured, but reliable analysis remains difficult. Organisation identifiers are fragmented, monetary fields have different accounting meanings, and many useful questions require several joins, filters, and aggregations. We build an auditable question answering framework from 166,277 Open Contracting Data Standard releases. The framework normalises the source records, represents questions as programs in a closed typed algebra, and executes each program with deterministic data conventions. It can also abstain when a question is ambiguous, unsupported, or not answered by the available records. The resulting benchmark contains 12,828 questions with executable gold programs. A second evaluator, written without shared execution code, reproduces 99.88% of 14,770 audited oracle cases.

Experiments separate the value of the executable scaffold from the value of model training. On development data, an untrained 8B model rises from a 31.5% retrieval baseline to 70.4% when routed through the scaffold. Task-specific training raises the strongest fully local system to 85.65% on a sealed 2,285-question test, compared with 69.76% for the cloud teacher used to propose training candidates. The ordering holds with a second 8B base model and on a sealed procurement challenge set, where the local planner reaches 78.31%. A transfer to WikiTableQuestions raises an untrained 22.51% baseline to 44.43% with answer-only supervision and 51.80% with gold programs. The transfer audit identifies program-language and representation coverage as the main remaining bottleneck.

1 Introduction

Large language models can interpret natural-language questions and produce fluent answers, but they remain unreliable when a task requires several linked operations. A model may retrieve the right records while applying the wrong relation, counting the wrong unit, or aggregating values that should remain separate. Structured query systems offer a different strength. Their entities, relations, and operations can be inspected and executed again. Their weakness is coverage, since they depend on a defined schema and a restricted query language. Reliable question answering over structured data therefore requires both flexible language understanding and a reproducible path from evidence to answer.

UK public procurement is a demanding setting for this problem. Contract notices contain buyers, suppliers, awards, dates, locations, and monetary values, which makes them suitable for factual, temporal, and compositional analysis. The records are not clean enough to query directly. One organisation can appear under many source identifiers and name variants. Values may be reported at tender, lot, award, and contract level, where some fields represent separate amounts and others repeat the same amount. A program can therefore be valid and executable while still computing the wrong quantity. Correctness depends on the program, the data conventions, and the link between the two.

Prior work addresses parts of this setting. Retrieval-augmented generation can supply relevant records, but it does not guarantee correct joins, counts, or aggregations. Neural-symbolic question answering maps language to executable programs, but is usually evaluated over curated schemas and stable execution environments (Cheng et al., 2023; Gu et al., 2023; Luo et al., 2024a). Methods such as STaR and rejection-sampling

fine-tuning learn from selected model outputs (Zelikman et al., 2022; Yuan et al., 2023; Gulcehre et al., 2023). These methods show that executable or answer-based feedback can support training. They do not resolve the full problem considered here, where the source semantics must first be audited, refusal is part of the task, and successful execution does not by itself establish that the program answers the question.

We build the task and system end to end. Five years of procurement releases are converted into a canonical data layer with explicit rules for entity identity, value provenance, and monetary additivity. Questions are compiled into a small typed algebra. A language model proposes the interpretation and program. Deterministic code then checks the program, grounds it to the data, and executes it. During training, sampled programs are retained only when their executed answers match the benchmark oracle. During deployment, no oracle is available or required. The final answer comes from execution, not free-form model generation.

The evaluation asks three questions. First, how much of the performance comes from the executable scaffold, and how much comes from task-specific model training? Second, does the resulting capability survive new program structures and question surfaces? Third, when the architecture is moved to another structured-data domain, which component limits performance? The experiments show that the scaffold establishes a strong floor, training both question understanding and planning improves generalisation, and cross-domain transfer is limited mainly by representation and program-language coverage.

Contributions. This work makes three contributions.

1. We construct an auditable procurement question-answering task with explicit data conventions, executable gold programs, scored abstention, and answer oracles checked by an independently implemented evaluator.
2. We develop a typed executable framework that separates language understanding from deterministic computation. The same framework supports program training from teacher and self-generated candidates without making the language model the source of the final answer.
3. We provide a controlled empirical decomposition across retrieval, executable scaffolding, local planning, sealed external evaluation, and table-domain transfer. The analysis identifies where performance comes from and where the closed language still limits coverage.

2 Related Work

Executable question answering. Semantic parsing and knowledge-base question answering translate natural language into logical forms that can be executed against structured knowledge. Recent systems improve coverage through in-context program generation, graph-constrained search, retrieval-based grounding, or tool use (Cheng et al., 2023; Gu et al., 2023; Luo et al., 2024a;b; Jiang et al., 2023). Table question-answering systems use SQL, Python, neural table encoders, or hybrid execution (Pasupat & Liang, 2015; Herzig et al., 2020; Yin et al., 2020; Liu et al., 2022; Jiang et al., 2022; Ye et al., 2023; Ni et al., 2023; Wang et al., 2024). These methods establish the value of executable intermediate representations. Our setting adds a problem that curated benchmarks usually remove. The source fields and entity identities are themselves uncertain, so execution is only meaningful after the data conventions have been specified and audited.

Learning from generated programs. STaR, rejection-sampling fine-tuning, and ReST generate candidate solutions, retain those that satisfy an acceptance rule, and train on the survivors (Zelikman et al., 2022; Yuan et al., 2023; Gulcehre et al., 2023). V-STaR learns a verifier from correct and incorrect solutions, while executable SQL systems use answer or execution feedback to build training data (Hosseini et al., 2024; Zhang et al., 2025). Weak-to-strong work studies when a student can generalise beyond imperfect supervision (Burns et al., 2024). We do not claim the generate-filter-train loop as a new method. We use it as one part of an executable system and study how it behaves when a program can compile and run while expressing the wrong analysis.

Procurement data, provenance, and refusal. Procurement knowledge graphs such as TheyBuyForYou support data integration, search, and risk analysis across public records (Soylu et al., 2022). Our focus is a

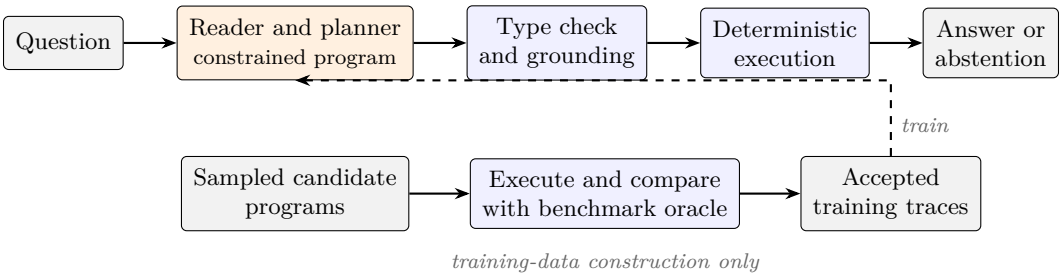

Figure 1: System overview. The deployment path ends with deterministic execution. Oracle comparison is used only when constructing training data.

measurable natural-language task rather than a graph resource alone. Each answer is tied to an executable program and grounded records. Questions that are ambiguous, unsupported by the data model, or matched by no records are represented explicitly. This connects provenance to the execution trace and makes failure detection part of the benchmark rather than an informal property of the interface.

3 Task and Method

3.1 Overview

Figure 1 separates deployment from data construction. At deployment time, the model reads a question and emits a program under constrained decoding. Deterministic components check, ground, and execute the program, then return an answer or abstention. During training-data collection, candidate programs are sampled at positive temperature and compared with the benchmark oracle after execution. Only matching candidates enter the training pool. Oracle access is therefore a property of the data pipeline, not the runtime system.

3.2 Data conventions and canonical representation

The source corpus contains 166,277 compiled OCDS releases from 2022 to 2026. A field audit identified two issues that determine many downstream answers. First, signed values are often stored in contract sub-records rather than the nominal award-value field. Second, most buyer and supplier references are platform account identifiers. One real organisation can therefore appear under many IDs. The Ministry of Defence appears under 77 platform identifiers in one year of the source data.

We convert the releases into a canonical representation with 215,221 award records and 131,502 organisations. Entity resolution is precision-first. Registry identifiers and safe deterministic variants are merged before normalised name and regional evidence are considered. Fuzzy matching produces review candidates but never writes a merge directly. Monetary values follow a fixed fallback rule and retain their source field and an additivity flag. The flag prevents repeated framework ceilings from being summed once for every linked award. The representation is materialised as typed columnar tables. This supports vectorised joins, unit tests, and a second evaluator that does not share execution code with the main system.

3.3 Program language and deterministic execution

Questions are represented as trees in a closed typed algebra. Seven value types and seventeen procurement operators cover filtering, projection, counting, guarded summation, grouping, ranking, scalar comparison, and set operations. Each node has an explicit input and output type. The complete inventory appears in Appendix A.

The algebra is narrower than SQL or Python, but it provides three useful properties. Membership is decidable by a recursive type checker. The grammar can be compiled into a JSON schema for token-level constrained decoding (Kwon et al., 2023). The operator set is small enough for a second evaluator to implement

independently. After a program is produced, the grounding layer resolves entities, fields, dates, and values to the canonical representation. The executor applies contract-level deduplication, exact decimal arithmetic, the additivity guard, empty-key exclusion, and deterministic tie breaking. It returns both the answer and the records used to compute it.

3.4 Planning, training, and abstention

The procurement system uses two learned stages. A reader extracts the requested answer type, question literals, and dependency structure. A planner compiles the question and brief into a program. This split was retained because it outperformed direct generation on procurement. The margin is quantified by a cross-base comparison. The fully local system on the weaker base outperforms the hybrid system on the stronger base by 5.30 points with a 95% interval of 3.62 to 6.97, which indicates that trained question reading contributes more than base capability on this task. The WikiTableQuestions transfer uses a fused reader-planner because fixed handoffs were weaker during table development. All local systems, including untrained baselines, use the same constrained decoder. This prevents output-format errors from dominating the comparison.

Training candidates are sampled from a cloud teacher or from the local student. A candidate is retained when it has a valid type structure, grounds to the question and data, executes successfully, and produces the oracle answer. The oracle does not author the candidate and is not included in the model input. The training ladder includes supervised fine-tuning, self-harvest with rejection sampling, and preference optimisation. Preference optimisation is not used in the primary systems because its near-miss negatives were unstable across the two bases.

Abstention is part of the output space. The system can return ambiguous, unsupported, or no-result when the question or evidence does not justify an answer. A repair step is available only after a typed failure. Refusals and meaningful empty results are final, since an earlier ungated repair design changed six correct refusals into incorrect answers.

3.5 Benchmark construction and oracle audit

The main benchmark is generated program first. Parameterised programs are instantiated and executed before a question surface is written, so every row is created with an executable gold program and answer. The benchmark contains 12,828 questions, including answerable, ambiguous, unsupported, and empty-result cases. Splits separate program structures rather than surface wording. Mechanical gates prevent plan overlap, duplicate identifiers, local sampling omissions, and multi-answer ambiguity.

A second evaluator reconstructs answers from the raw releases without shared execution code. Across 14,770 audited oracle cases, the two implementations agree in 99.88% of cases. All 18 remaining disagreements were inspected, and none affects a reported comparison. This audit measures implementation agreement under the declared data conventions. It does not remove uncertainty from the source records themselves.

4 Evaluation

Benchmarks. The main evaluation uses a sealed test of 2,285 procurement questions. PACS contains 922 rows from seven procurement-analysis families and tests whether the system depends on the surface generator used for the main benchmark. Its specification, question grammar, and logical-shape labels were frozen before the model was evaluated. Five isolation checks cover text, program hash, template identity, entity anchors, and logical shape. Each sealed intent is rendered through an independently written surface grammar, and naturalised rewrites pass deterministic checks of operation, negation, comparison direction, quantifier, and scope against the gold program's logical signature. WikiTableQuestions contains 22,033 crowd-written questions over 2,108 web tables (Pasupat & Liang, 2015). Development uses table-disjoint folds, and the official 4,344-row test is evaluated once under a recorded manifest.

Systems. Table 1 shows the five main procurement systems. The local models are Qwen3-8B and Llama-3.1-8B, trained with rank-64 LoRA under 4-bit QLoRA (Hu et al., 2022; Dettmers et al., 2023). All local

Table 1: Main procurement systems. All systems use the same data layer and executor.

System	Question reading	Program planning	Training
Cloud teacher	Cloud	Cloud	None
Llama hybrid	Cloud	Llama-3.1-8B	In-domain
Qwen hybrid	Cloud	Qwen3-8B	In-domain
Llama fully local	Llama-3.1-8B	Llama-3.1-8B	In-domain
Qwen fully local	Qwen3-8B	Qwen3-8B	In-domain

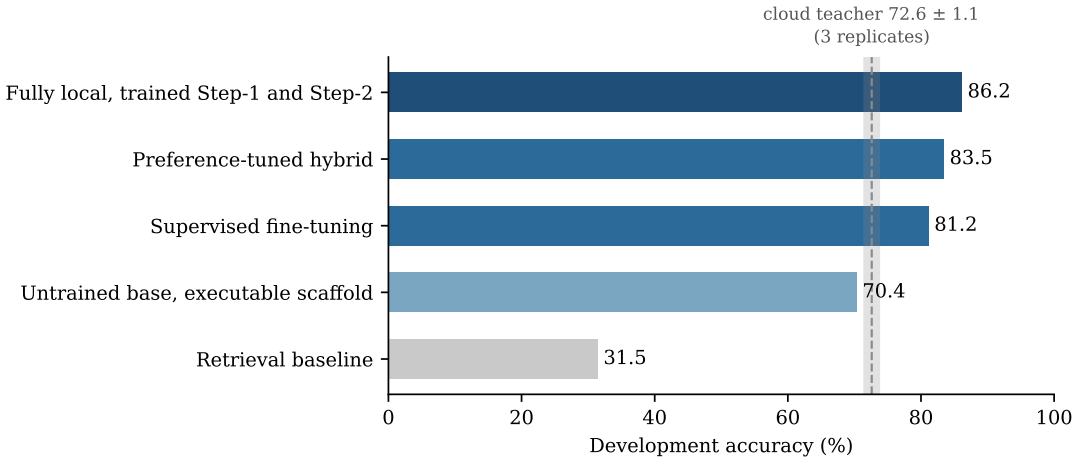

Figure 2: Development-set capability ladder. The executable scaffold establishes most of the gap over retrieval, while task-specific training adds further gains. The shaded band shows three cloud-teacher runs.

systems use the same executor and constrained decoder. The cloud teacher uses its provider API and cannot use the same token-level grammar constraint, so teacher comparisons are reported as deployable-system comparisons rather than equal-decoding model comparisons.

Protocol. Each primary evaluation uses one temperature-zero model call and no repair. Answerable questions use type-aware exact match against the audited oracle. Refusal questions use strict status match and a faithfulness-gated metric that checks whether every literal in the emitted program is supported by the question. Paired systems are compared with exact McNemar tests (McNemar, 1947). PACS intervals use cluster bootstrap over intent clusters. The cloud teacher cannot be seeded, so its variability was measured with three identical development runs at 72.6 ± 1.1 points, and no teacher comparison below three points rests on a single run. Each student is a single frozen training run. Paired significance therefore measures test-set disagreement and not training-seed variance.

5 Results

5.1 Where the procurement performance comes from

Figure 2 separates the executable scaffold from model training. The retrieval baseline reaches 31.5% on development. An untrained Qwen3-8B model routed through the typed program interface and executor reaches 70.4%, already close to the cloud teacher's replicate band. Supervised fine-tuning raises accuracy to 81.2%, and training both the reading and planning stages reaches 86.2%. The large first step shows that exhaustive computation and a fixed execution contract provide much of the initial gain. The remaining steps measure task-specific planning and question understanding.

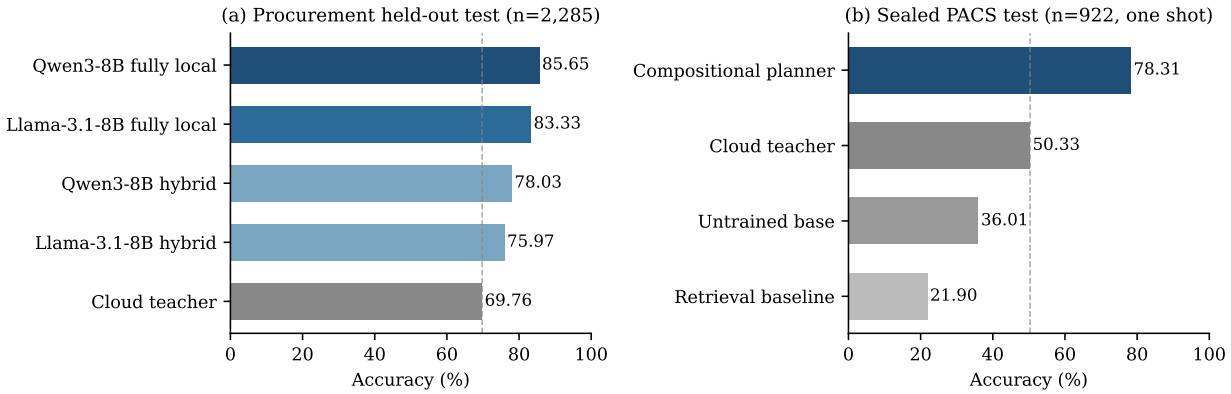

Figure 3: Main scoreboards. (a) Sealed procurement test. (b) Sealed PACS test. Dashed lines mark the cloud teacher in each panel.

A format diagnostic supports reading the ladder as planning quality rather than output discipline. Decoded without constraints on 100 development questions, the untrained base produces question-grounded plans with real entities in an invented format on 98 and matches the executable contract on 3, while the tuned models match the contract on 98 and 99. Fine-tuning therefore teaches conformance to the specific contract, and constrained decoding grants that contract to every system, so accuracy differences under the shared decoder measure planning.

The sealed test confirms the ordering. Figure 3(a) shows 69.76% for the cloud teacher, 75.97% and 78.03% for the hybrid systems, and 83.33% and 85.65% for the fully local systems. The strongest local system is 15.89 points above the teacher, with a 95% paired interval of 14.07 to 17.70 and exact McNemar $p < 10^{-14}$. The same ordering holds with the second base, which reduces the chance that the result is specific to one model family. Fully local systems also outperform the hybrids on both bases. This indicates that task-specific question reading contributes beyond local program planning.

5.2 Distribution shift and independent procurement questions

Figure 4(a) compares development and test. The hybrid systems lose 5.5 and 7.1 points, while the fully local systems lose 0.5 and 0.9. On the composite-hard development slice, the Qwen hybrid loses 7.7 points and the fully local Qwen gains 0.8. The teacher also declines across its three runs. These results are consistent with the fully local system depending less on the cloud reader and the development surface distribution. They do not by themselves establish a causal robustness mechanism.

PACS tests new question surfaces and logical exposure. The frozen local planner reaches 78.31% strict accuracy on 922 sealed rows and 80.00% on the answerable subset, compared with 50.33% for the cloud teacher. The teacher API lacks the local constrained decoder, and 23.1% of its rows fail on format, so the margin is not presented as a decoding-neutral model comparison.

The PACS audits in Figure 5 separate depth from novelty. A row is irreducible when deleting any predicate from its gold program changes the answer. This holds for 42.7% of answerable rows, and the planner scores 78.7% on that subset, close to the 80.0% answerable result. Intent shapes not shown during training reduce accuracy to 68.5%. The system therefore transfers new compositions of familiar constructs more reliably than entirely new logical forms. The repair result in the same figure is a development variant and is not part of the sealed headline.

5.3 Transfer to WikiTableQuestions

The transfer retains the typed execution architecture and replaces the procurement data adapter with table views, a column-aware value linker, and one row-preserving arg-extremum operator. Table 2 reports the

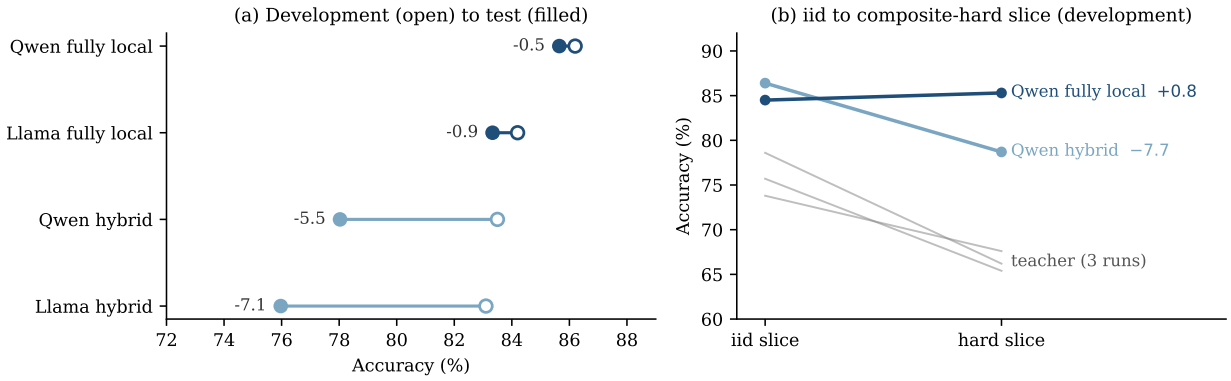

Figure 4: Robustness analyses. (a) Development-to-test movement. (b) IID to composite-hard movement on development data.

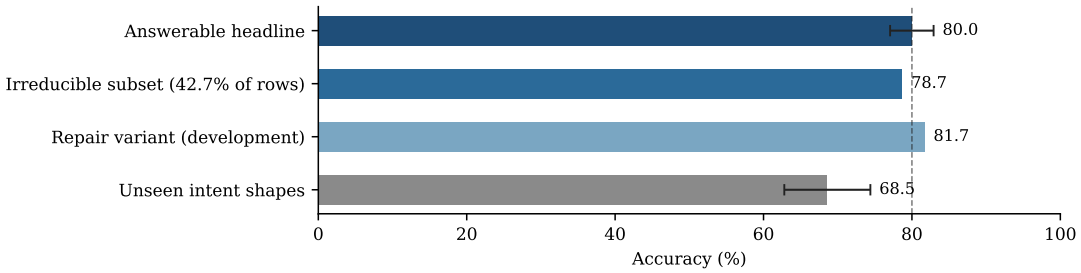

Figure 5: PACS audits. The irreducible subset remains close to the answerable headline, while unseen intent shapes produce the largest measured loss. The repair arm is a development-only variant.

official test results. The procurement checkpoint carries a 4.82-point zero-shot gain over the untrained base. Answer-only training reaches 44.43%, and gold-program training reaches 51.80%. The answer-only result is comparable with classical weakly supervised parsers, but both trained systems remain below recent specialised table-QA methods (Cheng et al., 2023; Ye et al., 2023; Wang et al., 2024). The experiment is used to study portability and bottlenecks, not to claim state of the art.

A differential audit against 11,276 SQUALL SQL programs (Shi et al., 2020) locates the remaining gap. The original algebra expresses 53.9% of the reference programs. Compiler closure and typed loader views recover 730 additional programs and raise coverage to 61.0%. Of these, 562 come from translator closure, 154 from loader views, and 14 require both. The remaining 39.6% lie outside the language. Figure 6 shows the resulting attribution. Translated in-coverage programs match the reference denotation in 92–93% of cases, and execution recovers the benchmark answer in 94.1% of those cases. The reference programs are themselves imperfect, reproducing the benchmark answer in 88.9% of cases, so the audit treats them as a noisy reference rather than an absolute standard. This pattern points to coverage rather than executor correctness as the main transfer limit.

The coverage account is also tested by intervention. After closure, the answer-only development model rises from 31.7% to 40.0%, and the gold-program model rises from 40.0% to 51.0%. Figure 7 shows that both supervision regimes improve when the representation and compiler cover more of the task. This is the clearest evidence that the transfer gap is not solely a planning-model problem.

Table 2: WikiTableQuestions official-test accuracy.

System	Accuracy
Untrained local base	22.51
Procurement checkpoint, zero-shot	27.33
Answer-only training	44.43
Gold-program training	**51.80**

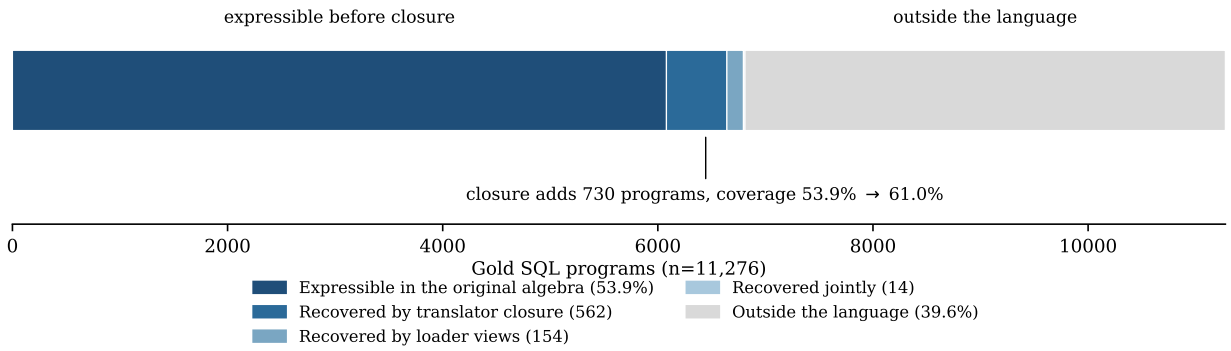

Figure 6: Attribution of WikiTableQuestions program coverage. Compiler and loader closure raise measured coverage from 53.9% to 61.0%, while 39.6% remains outside the language.

6 Discussion and Limitations

The results support three conclusions. First, a deterministic execution contract changes the problem substantially. An untrained local model becomes competitive with the cloud teacher once both operate through a typed program interface and executor. Second, task-specific training still matters. The two fully local systems outperform their hybrid counterparts and degrade less from development to test. Third, portability is bounded by what the data adapter, compiler, and algebra can represent. The WikiTableQuestions audit measures this boundary directly and shows that widening coverage improves both supervision regimes.

The teacher comparison is task-specific. The student receives in-domain training, while the teacher is evaluated as a zero-shot deployment. The result shows that the full local system is more accurate on the defined task, not that an 8B model has greater general capability. PACS strengthens the external validity claim, but it is still a controlled benchmark rather than a sample of real analyst questions.

Five limitations remain. The main benchmark is program-first and its question surfaces are generated under style controls. Each student result comes from one training run, so training-seed variance is not estimated. The teacher and local systems do not share identical decoding constraints. The closed algebra rejects questions outside its supported operations, and unseen intent shapes remain a clear weakness. Finally, the study does not simulate arbitrary deletion of KG facts. Its refusal tests cover missing or unsupported evidence, but not every form of data incompleteness.

The candidate-selection experiment in Appendix C provides an additional negative result. An LLM selector does not improve over the first accepted candidate, while a small deterministic scorer based on execution features gains 2.0 points. This finding concerns best-of-k selection only. It is not evidence that all model-based verification is ineffective.

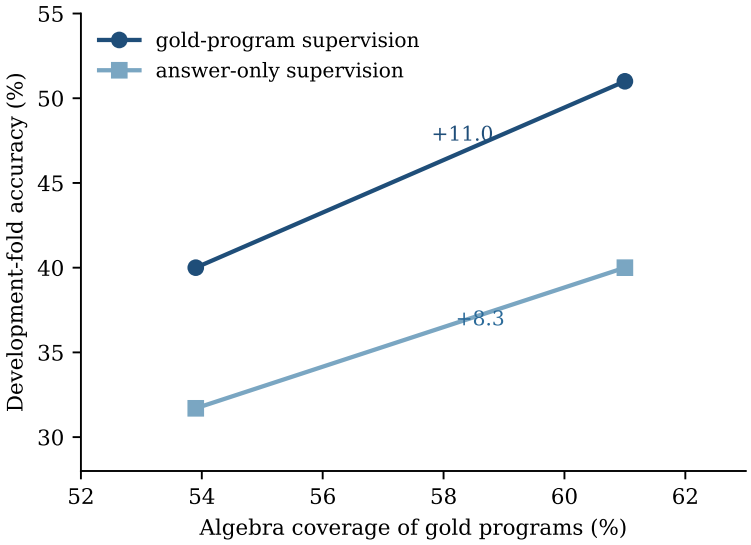

Figure 7: Development accuracy before and after WikiTableQuestions coverage closure. Both answer-only and gold-program training improve after the same representation and compiler intervention.

7 Conclusion

We presented an auditable question-answering framework for UK public procurement. The framework combines explicit data conventions, typed program generation, deterministic execution, abstention, and independent oracle audit. The experiments show that the executable scaffold establishes a strong base, local training of both reading and planning improves generalisation, and the result transfers to independent procurement questions. The table-domain study shows that representation and program-language coverage are the main remaining limits.

References

Collin Burns, Pavel Izmailov, Jan Hendrik Kirchner, Bowen Baker, Leo Gao, Leopold Aschenbrenner, Yining Chen, Adrien Ecoffet, Manas Joglekar, Jan Leike, Ilya Sutskever, and Jeff Wu. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. In *International Conference on Machine Learning (ICML)*, 2024.

Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. Binding language models in symbolic languages. In *International Conference on Learning Representations (ICLR)*, 2023.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Yu Gu, Xiang Deng, and Yu Su. Don't generate, discriminate: A proposal for grounding language models to real-world environments. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (ReST) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.

Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Martin Eisenschlos. TaPas: Weakly supervised table parsing via pre-training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

Arian Hosseini, Xingdi Yuan, Nikolay Malkin, Aaron Courville, Alessandro Sordoni, and Rishabh Agarwal. V-STaR: Training verifiers for self-taught reasoners. In *First Conference on Language Modeling (COLM)*, 2024.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Wayne Xin Zhao, and Ji-Rong Wen. StructGPT: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Zhengbao Jiang, Yi Mao, Pengcheng He, Graham Neubig, and Weizhu Chen. OmniTab: Pretraining with natural and synthetic data for few-shot table-based question answering. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2022.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, Weizhu Chen, and Jian-Guang Lou. TAPEX: Table pre-training via learning a neural SQL executor. In *International Conference on Learning Representations (ICLR)*, 2022.

Haoran Luo, Haihong E, Zichen Tang, Shiyao Peng, Yikai Guo, Wentai Zhang, Chenghao Ma, Guanting Dong, Meina Song, Wei Lin, Yifan Zhu, and Anh Tuan Luu. ChatKBQA: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models. In *Findings of the Association for Computational Linguistics: ACL*, 2024a.

Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations (ICLR)*, 2024b.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida I. Wang, and Xi Victoria Lin. LEVER: Learning to verify language-to-code generation with execution. In *International Conference on Machine Learning (ICML)*, 2023.

Panupong Pasupat and Percy Liang. Compositional semantic parsing on semi-structured tables. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2015.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Tianze Shi, Chen Zhao, Jordan Boyd-Graber, Hal Daumé III, and Lillian Lee. On the potential of lexico-logical alignments for semantic parsing to SQL queries. In *Findings of the Association for Computational Linguistics: EMNLP*, 2020.

Ahmet Soylu, Oscar Corcho, Brian Elvesæter, Carlos Badenes-Olmedo, Tom Blount, Francisco Yedro Martínez, Matej Kovacic, Matej Posinkovic, Ian Makgill, Chris Taggart, Elena Simperl, Till C. Lech, and Dumitru Roman. TheyBuyForYou platform and knowledge graph: Expanding horizons in public procurement with open linked data. *Semantic Web*, 13(2):265–291, 2022.

Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. Chain-of-table: Evolving tables in the reasoning chain for table understanding. In *International Conference on Learning Representations (ICLR)*, 2024.

Yunhu Ye, Binyuan Hui, Min Yang, Binhua Li, Fei Huang, and Yongbin Li. Large language models are versatile decomposers: Decomposing evidence and questions for table-based reasoning. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2023.

Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. TaBERT: Pretraining for joint understanding of textual and tabular data. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Jipeng Zhang, Haolin Yang, Kehao Miao, Ruiyuan Zhang, Renjie Pi, Jiahui Gao, and Xiaofang Zhou. ExeSQL: Self-taught text-to-SQL models with execution-driven bootstrapping for SQL dialects. In *Findings of the Association for Computational Linguistics: EMNLP*, 2025.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. LlamaFactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (System Demonstrations)*, 2024.

A Operator Algebra

Programs are trees over seven value types. RECORDS is a subset of the record universe, VALUES is a distinct set of scalar values, GROUPS maps group keys to numbers, and RANKING is an ordered list of key-value pairs. NUMBER, VALUE, and BOOL are scalar types. Program depth is capped at 16 and node count at 64. Each checker failure contains a typed diagnostic and a path into the program tree.

B Audit and Supplementary Results

Entity resolution. The source contains 204,711 raw organisation aliases. Resolution proceeds through registry identifiers, safe deterministic variants, normalised name and region, and finally name-only evidence. A total of 255 borderline cases were placed in a review file, and 82,013 singletons were left unmerged. Similarity matching never writes a merge directly.

Benchmark gates. Every generated program is re-executed against the full graph before its question is admitted. The build rejects local sampling omissions, multi-answer ambiguity, plan overlap, duplicate identifiers, and row-count changes. A position-aware mechanical detector checks whether paraphrases preserve the logical signature. Nineteen of the 2,285 final-test rows contain phrases also seen in refusal traps. Removing them changes every headline by at most 0.21 points and changes no ranking.

PACS isolation. Five zero-overlap checks hold between training and sealed evaluation at the level of question text, logical signature, template identity, entity anchors, and program hash. A 93-row human audit took place before unsealing. It found one systematic boolean error affecting 46 rows, which was corrected by offline re-execution before any model score was read.

Table 3: Closed operator inventory. The WikiTableQuestions transfer adds the row-preserving `extreme_rows` node.

Node	Signature	Function
`filter`	predicates → RECORDS	filter records
`values`	RECORDS → VALUES	project distinct values
`count`	RECORDS → NUMBER	count records
`size`	VALUES → NUMBER	set cardinality
`sum`	RECORDS → NUMBER	guarded money sum
`exists`	RECORDS → BOOL	test non-emptiness
`select`	RECORDS → VALUE	unique field lookup
`extreme`	RECORDS → VALUE	record argmin or argmax
`groupby`	RECORDS → GROUPS	grouped count or sum
`argext`	GROUPS → VALUE	extremum group key
`top`	GROUPS → RANKING	top-k groups
`num`	literal → NUMBER	numeric literal
`combine`	NUMBER^2 → scalar	arithmetic or comparison
`vcompare`	VALUE → BOOL	scalar comparison
`setop`	VALUES^2 → VALUES	set operation
`gcombine`	GROUPS^2 → GROUPS	combine grouped values
`keys_where`	GROUPS → VALUES	thresholded group keys
`extreme_rows`	RECORDS → RECORDS	row-preserving arg-extremum

Training ladder. Table 4 reports development accuracy and data-engine acceptance rates. Acceptance rate is the fraction of sampled answerable candidates that pass the data pipeline. It is not system accuracy. Acceptance rises across self-harvest rounds while the evaluation gain of the second round is 0.4 points and not significant, so the loop is reported at its measured ceiling rather than iterated further.

Table 4: Development ladder and candidate acceptance rates.

Arm	Qwen3-8B	Llama-3.1-8B
Zero-shot scaffolded	70.4	60.0
Supervised fine-tuning	81.2	83.1
Rejection-sampling fine-tuning	81.5	82.7
Preference optimisation	83.5	77.3
Fully local, best arm	86.2	84.2
Teacher candidate acceptance	65.5	
Self-harvest acceptance, round 1	76.6	78.1
Self-harvest acceptance, round 2	86.3	–

C Candidate Selection

Four candidate programs are sampled per development question after the basic program checks. A perfect selector would gain about five points over choosing the first accepted candidate. The deterministic scorer uses only execution features such as empty results, zero-row filter probes, and answer-kind mismatches. It gains 2.0 points (Figure 8). A faithful LLM selector gains 0.0, and an immediate LLM score loses 0.7. This experiment evaluates candidate ranking and not the main single-call system.

D Reproduction Details

Students are Qwen3-8B (Qwen Team, 2025) and Llama-3.1-8B. Both are adapted with rank-64 LoRA under 4-bit QLoRA quantisation in LLaMA-Factory (Hu et al., 2022; Dettmers et al., 2023; Zheng et al., 2024). Serving uses vLLM with guided JSON decoding compiled from the algebra grammar (Kwon et al., 2023).

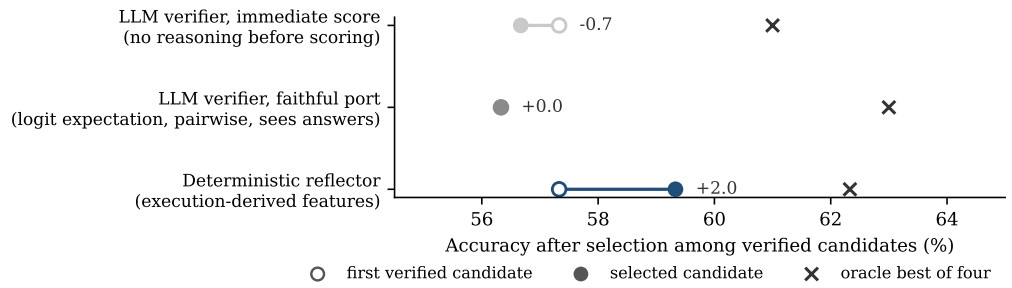

Figure 8: Candidate selection on development data. The deterministic scorer is the only selector that improves over the first accepted candidate. Crosses show the oracle best of four.

Candidate collection uses positive-temperature sampling, while evaluation uses one temperature-zero call. Training and evaluation ran on single 80GB A100 and 96GB H100 devices. The WikiTableQuestions official run used a recorded manifest containing the commit hash, file checksums, decoder settings, and adapter identity. An experiment ledger records every run behind this paper, and any rate-style number enters the ledger only after ten deterministically sampled raw transcripts, passes and failures both, have been read by a human. Prompts, manifests, tests, and the experiment ledger are released with the code.